

What's Actually Happening Inside a Language Model? A Deep Dive into Mechanistic Interpretability, part 1

What's Actually Happening Inside a Language Model? A Deep Dive into Mechanistic Interpretability, part 1

By Subhanga Upadhyay — MS CS 2026, Tufts University / AI Systems & Mechanistic Interpretability

There's a question that haunts me: **we built these systems, so why can't we explain what they're doing?**

Over the past few weeks, I've been going deep on mechanistic interpretability, not just reading papers, but running experiments on Gemma 2B, working through ARENA's interpretability curriculum, and building intuition from first principles. This post is a distillation of what I've learned, grounded in numbers from real experiments.

Let me take you from hardware constraints all the way to circuit-level findings.

Part 1: Before We Interpret Anything: Understanding What Lives in Memory

As I ran experiments on this topic, I realized I didn't actually know *what* I was fitting in my GPUs! It seemed to me that this was not trivia — it determined what I could study and do on a HPC, so I needed to know this.

The VRAM equation breaks down into four components:

Component	When it applies
Model weights	Always
Gradients	Training only
Optimizer state	Training only (Adam = $2\times$ weight size)
Activations	Training (backprop needs them); negligible at inference

Precision determines weight size:

- fp16: 2 bytes/parameter
- int8: 1 byte/parameter
- (quantized) int4 (NF4): 0.5 bytes/parameter

Key numbers on an A100 80GB:

Model	Precision	VRAM (weights only)	Inference?
Gemma 2B	fp16	~4 GB	Yes — Comfortable
LLaMA 8B	fp16	~16 GB	Yes — Fine
Gemma 32B	fp16	~64 GB	Yes — Tight but fits
LLaMA 70B	fp16	~140 GB	No — Need 4-bit
LLaMA 70B	int4	~35 GB	Yes — With quantization

Why does inference use so much less memory than training?

During inference, you only need the weights and a small KV cache (negligible for short sequences), which is exactly the case for my experimentations, an ant compared to LLM providers who have to deal with arbitrarily large text that people apply at inference. Since the kv cache scales with `seq_len` (the input token sequence), this is a giant problem for those folks which is why tons of research is poured into KV cache optimization, and why the paper TurboQuant by Google was such a big deal!

But for my experiments, the inference input tokens are always constrained (typically < 100 words), so this is not a problem at all.

In summary:

- During training: you need weights + gradients (same size as weights) + Adam optimizer state ($2\times$ weight size)¹ + stored activations for backprop. A 16GB model becomes a ~64GB training job before you've touched the data.

¹you can use any optimizer that you want! That is the crux of this finding; there are optimizers particularly focusing in saving VRAM memory, so we can shop around. Of course, these things have a tradeoff because Adam is as SOTA as you get, but there's that.

This is why **QLoRA** is such a breakthrough. You quantize the frozen weights to 4-bit (halving the weight footprint), then train only low-rank adapter matrices (LoRA). Only the tiny adapter parameters need gradients and optimizer state. Result: you can fine-tune LLaMA 70B on a single A100 80GB. The math works.

Part 2: Why LoRA Works — And Why It Shouldn't Surprise You

Here's the counterintuitive result: you can freeze 99.997% of a model's weights, update only ~4.7M parameters out of 175B (GPT-3 scale), and match full fine-tuning performance on most benchmarks.

Why?

The low-rank hypothesis. When you fine-tune a pretrained model, the weight update ΔW is intrinsically low-rank. Even though ΔW lives in a space of millions of dimensions, the actual information content of the adaptation lives in a tiny subspace — rank 4, 8, or 16.

Formally, LoRA approximates:

$$W_{\text{new}} = W_{\text{pretrained}} + \Delta W \approx W_{\text{pretrained}} + BA$$

where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$, with $r \ll \min(d, k)$.

The intuition: pretraining already built a rich representation of language. Fine-tuning doesn't rewrite that: it *steers* it. Steering is a low-rank operation. You're amplifying existing directions in weight space, not building new ones.

Aghajanyan et al. (2020) measured the intrinsic dimensionality of fine-tuning tasks and found that 90% of performance can be recovered using **hundreds to low thousands** of parameters, even for models with hundreds of millions of weights. The high-dimensional space is massively redundant.

Part 3: Activation Patching: the Core Tool of Mechanistic Interpretability

Now we get to the interesting part.

The fundamental question of mechanistic interpretability: which specific components of a neural network are causally responsible for a specific behavior?

Activation patching answers this through controlled substitution experiments.

The setup:

Take two prompts, clean and corrupted, that differ in exactly one entity:

Clean: "The Roman god Jupiter is the Greek god ___" → Zeus

Corrupt: "The Roman god Ares is the Greek god ___" → Mars

Run both through the model. Cache all intermediate activations.

The experiment: take a single activation from the corrupt run (e.g., the residual stream at layer 6, at the position of the entity token), patch it into the clean run at the same position, and measure what happens to the output.

The metric: logit difference:

$LD = \text{logit}(\text{Zeus}) - \text{logit}(\text{Mars})$

- Clean run: LD is large and positive (model correctly prefers Zeus)
- Corrupt run: LD is large and negative (model correctly prefers Mars)

When you patch a component, you measure:

$$\Delta LD = LD_{\text{after_patch}} - LD_{\text{clean}}$$

A large negative ΔLD means the patched component was load-bearing for the correct answer.

To make this comparable across prompt pairs, we normalize:

$\text{metric} = (\text{current_LD} - \text{corrupted_LD}) / (\text{clean_LD} - \text{corrupted_LD})$

- 0 → component had no effect (null hypothesis holds)
- 1 → component fully recovered clean performance
- Everything in between is meaningful signal

The null hypothesis: patching this component does nothing. The corrupt run stays broken. When you reject the null, you've found a causally relevant piece of the circuit.

Part 4: The Three-Dimensional Patching Space

Every component in a transformer produces an activation at every token position at every layer. This gives you a 3D space to explore:

	"The"	"capital"	"of"	"France"	"is"	"___"	
Layer 0:	[*]	[*]		[*]	[*]	[*]	[*]
Layer 1:	[*]	[*]		[*]	[*]	[*]	[*]
...							
Layer 17:	[*]	[*]		[*]	[*]	[*]	[*]

Dimension 1 — Layer: which row to patch

Dimension 2 — Token position: which column

Dimension 3 — Component: residual stream, attention output, or MLP output at that cell

Each experiment fixes some dimensions and sweeps others. This is exactly the methodology I used in BizzaroWorld (my mechanistic interpretability study on Gemma 2B):

- **Experiment 1:** Fixed column (final token position), swept all rows → found late layers 15–17 bear causal load. Mean worst layer = 16.3 across all three selection modes.
- **Experiment 3:** Fixed different column (entity token position), swept all rows → found early layers 0–14 store entity information. Mirror image of Experiment 1.
- **Experiment 2A:** Fixed both column AND rows (layers 15–17), swept the third dimension (sublayers) → found residual stream dominates 60× over attention, 18× over MLP.

Part 5: The IOI Task: a Benchmark Circuit

The **Indirect Object Identification (IOI)** task is the canonical benchmark for mechanistic interpretability work. The prompt is:

"When John and Mary went to the shops, John gave the bag to ___"

The correct completion is "Mary" (the indirect object). The corrupted version swaps the names:

"When John and Mary went to the shops, Mary gave the bag to ___"

This task was chosen because: 1. It has a clear, unambiguous correct answer 2. It involves genuine multi-step reasoning (track two names, identify which is the IO) 3. GPT-2 Small solves it reliably, making it tractable to study

resid_pre Activation Patching

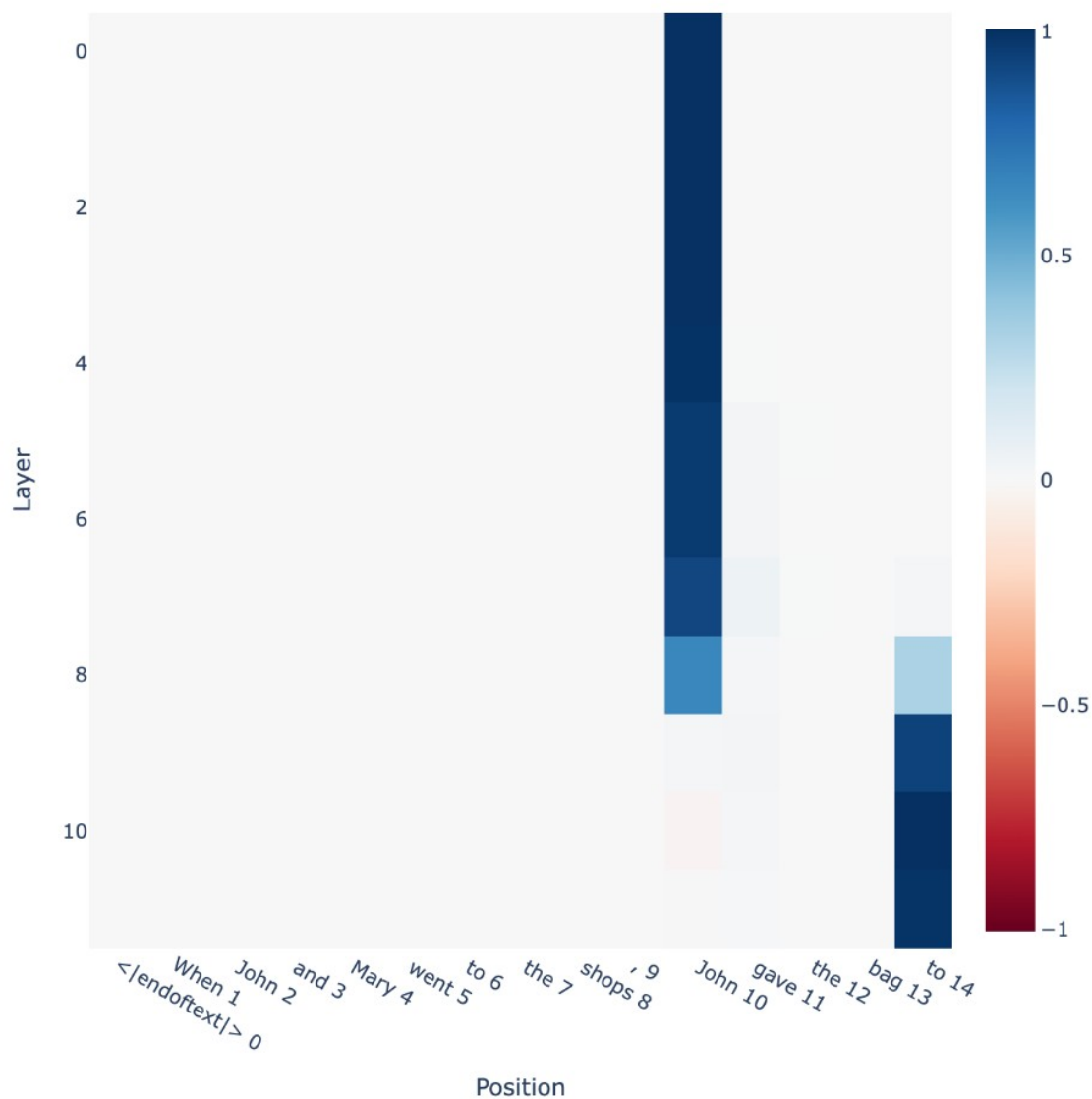


Figure 1: `resid_pre` activation patching heatmap — layers $\times$ token positions, color = `ioi_metric` score

The heatmap reveals immediately which (layer, position) cells, when patched from the clean run into the corrupt run, recover the correct answer. The signal concentrates in specific layers at specific token positions — not uniformly distributed.

Denoising vs. Noising: two Complementary Questions

Mode	Direction	Question	0 means	1 means
Denoising	Clean $\rightarrow$ corrupt	Is this component <i>sufficient</i> ?	Component irrelevant	Fully recovers performance
Noising	Corrupt $\rightarrow$ clean	Is this component <i>necessary</i> ?	Component irrelevant	Fully destroys performance

A component that is both necessary and sufficient is the core of the circuit. Components that are sufficient but not necessary indicate redundancy. Components that are necessary but not sufficient indicate distributed computation, the finding I keep hitting on Gemma2B for factual recall.

Part 6: What This Means: and What Comes Next

The three-phase circuit is a clean, falsifiable claim about how Gemma 2B retrieves facts. It says:

1. Facts are stored as directions in the residual stream at early-to-mid layers
2. Attention heads collectively route that signal to the prediction position
3. Late layers read out the answer with minimal additional computation

What I haven't done yet (next experiments):

- **Path patching (Experiment 5):** sharpens the distributed routing finding into a directed circuit graph — which components causally influence which others, and in what order
- **CMAF:** cross-model activation patching between Gemma 2B base and instruction-tuned to see how fine-tuning changes the circuit
- **Cross-architecture replication:** do LLaMA 8B, Mistral 7B, and Qwen 2.5 7B show the same three phases? If yes, this is a universal finding about transformer factual recall

The bigger question this points toward:

If factual recall follows a clean three-phase circuit, what does that tell us about how to diagnose hallucinations? If a model hallucinates, is the storage phase corrupted? The routing phase? Or is the readout reading from a correctly-stored but wrong fact?

Mechanistic interpretability gives us the tools to ask — and potentially answer — that question precisely.

Resources

- ARENA curriculum: arena.education — hands-down the best structured path into transformer interpretability
- Wang et al. (2022): “Interpretability in the Wild” — the IOI paper that established path patching
- Conmy et al. (2023): ACDC — automated circuit discovery
- TransformerLens: github.com/neelnanda-io/TransformerLens — the toolkit that makes all of this tractable
- <https://arxiv.org/html/2402.14811v1>

If you're working on mechanistic interpretability, MLOps observability, or production ML reliability — I'd love to connect. I'm building Scorpion Labs, an AI consulting practice focused on making ML systems interpretable and reliable in production.

Tags: #MachineLearning #MechanisticInterpretability #AI #LLM #MLSystems
#DeepLearning #Transformers #AIRsearch